

**DESIGN AND IMPLEMENTATION OF A SCIENTIFIC CALCULATOR
FEATURING NUMBER BASE CONVERSIONS, COMPLEMENTS,
GRAY CODE, AND OHM'S LAW APPLICATIONS**



*An Internship Project Report submitted in the partial fulfilment of the
requirements for the degree of*

BACHELOR OF SCIENCE

Submitted By

Deep Jyoti Boruah

Registration No. 23SU20069

Enrolment No. 991992310510

Under the Supervision of

Shri Manab Kalita

Scientist – F/ECIC

Under the Guidance of

Mr. Jyotirmoy Deka

IT Faculty

National Institute of Electronics & Information Technology, Jorhat Extension Centre

MES Gate, Chaliha Gaon, Rowriah, Jorhat, Assam - 785004

Department Of Computer Science

Sibsagar University

Sivasagar, Assam, 785665

ABSTRACT

This project report details the design and implementation of a comprehensive calculator application developed using Python. The primary objective was to create a versatile computational tool capable of performing standard arithmetic operations alongside advanced functionalities crucial in digital electronics and computer science. Existing conventional calculators often lack integrated support for seamless number base conversions and direct digital logic operations, necessitating the use of multiple specialized tools.

The developed application addresses this gap by providing robust capabilities for inter-conversion between binary, octal, decimal, and hexadecimal number systems. Furthermore, it incorporates essential digital logic operations, including 1's and 2's complements, and Gray code conversions, crucial for understanding and manipulating digital data. Expanding its utility into basic electrical engineering, the calculator also features a dedicated module for calculating resistance using standard colour codes and for applying Ohm's Law.

Utilizing Python's versatile libraries for numerical computations and graphical user interface development, the application offers an intuitive and user-friendly interface. The methodology involved modular design, separating functionalities into distinct components for base conversions, logical operations, and electrical calculations, ensuring maintainability and scalability. Extensive testing was conducted to validate the accuracy and reliability of all implemented functions across various input scenarios.

The successful implementation of this calculator demonstrates the feasibility of consolidating diverse mathematical, logical, and electrical engineering functionalities into a single, accessible application. This tool serves as an efficient resource for students, educators, and professionals, streamlining complex calculations and enhancing understanding of fundamental concepts in digital systems and basic electronics.

DECLARATION

I, **DEEP JYOTI BORUAH**, bearing **University Registration No. 23SU20069** and **Enrolment No. 991992310510**, a students of **B.Sc. (Honours) 5th Semester** from the **Department of Computer Science, Sibsagar University** hereby declare that the internship project report entitled "**Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications**" is an authentic record of our own work carried out during the period of **July 7th, 2025** to **25th July, 2025**, as a part of our academic curriculum at **National Institute of Electronics & Information Technology (NIELIT), Jorhat Extension Centre** under the guidance of our supervisors **Shri Manab Kalita** and **Mr. Jyotirmoy Deka**.

I further declare that the work presented in this internship report has not been submitted by us for the award of any other degree or diploma in this or any other Institution. All information and data used from other sources have been duly acknowledged in the report.

This report is submitted in partial fulfilment of the requirements under the Four Years Undergraduate Programme (FYUGP) of Sibsagar University.

Date: 11th November 2025

Place: Sibsagar University, Sivasagar, Assam

With Regards
Deep Jyoti Boruah
Registration No. 23SU20069
Department of Computer Science
Sibsagar University



राष्ट्रीय इलेक्ट्रॉनिकी एवं सूचना प्रौद्योगिकी संस्थान (भा.इ.सू.प्रौ.सं)

National Institute of Electronics & Information Technology (NIELIT)

(इलेक्ट्रॉनिकी एवं सूचना प्रौद्योगिकी मंत्रालय, भारत सरकार की एक स्वायत्त वैज्ञानिक संस्था)

(An Autonomous Scientific Society of Ministry of Electronics and Information Technology (MeitY), Govt. of India)

गुवाहाटी केंद्र/ Guwahati Centre

वित्तीय भवन, असम वित्तीय निगम भवन, प्रथम एवं द्वितीय तल, मो. शाह मार्ग, पल्टन बाजार, गुवाहाटी -781008

Vittiya Bhavan, Assam Financial Corporation Building, 1st & 2nd Floor, Md. Shah Road, Paltan Bazar, Guwahati – 781008

दूरभाष/Phone: +91-361-2131568(WLL), 2730269(EPBAX) ई-मेल/E-mail : guwahati@nielit.gov.in

वेबसाइट/Website : <https://www.nielit.gov.in/guwahati>

CERTIFICATE

This is to certify that this report on Python based project entitled " **Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications** " is based on the academic and other work performed during the Internship at **National Institute of Electronics & Information Technology, Jorhat Extension Centre** from 7th July, 2025 to 25th July, 2025. This is a Bonafide work that has been carried out by **Mr. Deep Jyoti Boruah (Internship Regn. No. GW03PYT2507022)** bearing **University Registration No. 23SU20069** and **Enrolment No. 991992310510**, a student of **B.Sc. (Honours) 5th Semester** from the **Department of Computer Science, Sibsagar University**, under our supervision and guidance.

Date: 25th July 2025

Place: Jorhat, Assam


(Mr. Jyotirmoy Deka)

IT Faculty

NIELIT, Jorhat Extension Centre


(Shri Manab Kalita)

Scientist-F/Centre In-Charge

NIELIT, Jorhat Extension Centre

मुख्यालय: नाइलिट भवन, प्लॉट नंबर 3, पी.एस.पी. पॉकेट, सेक्टर 8, द्वारका, नई दिल्ली - 110077

Headquarters: NIELIT Bhawan, Plot No. 3, PSP Pocket, Sector - 8, Dwarka, New Delhi - 110077

दूरभाष/Phone: 91-11-24363330-1-2, फैक्स/Fax: 91-154363335, Website: <https://www.nielit.gov.in/>



DEPARTMENT OF COMPUTER SCIENCE

SIBSAGAR UNIVERSITY

P.O. Joysagar-785665, Sivasagar, Assam
Email : compscience@sibsagaruniversity.ac.in

Ref. No.

Date: 11th November 2025

CERTIFICATE

This is to certify that this report on Python based project entitled " **Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications** " is based on the academic and other work performed during the Internship at **National Institute of Electronics & Information Technology, Jorhat Extension Centre** from 7th July, 2025 to 25th July, 2025. This is a Bonafide work that has been carried out by **Mr. Deep Jyoti Boruah, (Internship Regn. No. GW03PYT2507022)** bearing **University Registration No. 23SU20069** and **Enrolment No. 991992310510**, a student of **B.Sc. (Honours) 5th Semester** from the **Department of Computer Science, Sibsagar University**, under our supervision and guidance.

Date: 11th November 2025

Place: Sibsagar University, Sivasagar, Assam

(Dr. Ranjan Sarmah)

Head of the Department

Department of Computer Science

Sibsagar University

ACKNOWLEDGEMENTS

I would like to express our sincere gratitude to all those who provided invaluable support and guidance throughout the duration of this project, entitled "**Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications.**"

First and foremost, we extend our deepest appreciation to **Mr. Jyotirmoy Deka, IT Faculty, NIELIT, Jorhat Extension Centre** for his invaluable guidance, whose profound knowledge, insightful suggestions, and continuous encouragement were instrumental in the successful completion of this work. Their unwavering support and guidance significantly shaped the direction and quality of this project.

I am also profoundly grateful to **Shri Manab Kalita, Scientist-F, Centre In-charge, NIELIT, Jorhat Extension Centre**, for providing us the necessary facilities, resources, and a conducive environment that enabled us to carry out this project effectively. Their administrative support was highly valuable.

My sincere thanks also go to **Dr. Ranjan Sarmah, Head, Department of Computer Science, Sibsagar University** and all the faculty members of **Department of Computer Science, Sibsagar University** for their academic support and for fostering an environment conducive to learning and innovation.

I would also like to thank my friends and family for their continuous encouragement and moral support throughout this journey.

Finally, I am thankful to the Almighty for providing me with the strength and determination to complete this project.

Date: 11th November 2025

Place: Sibsagar University, Sivasagar, Assam

With Regards
Deep Jyoti Boruah
Registration No. 23SU20069
Department of Computer Science
Sibsagar University

TABLE OF CONTENTS

CONTENTS	PAGE NO
❖ TITLE PAGE	1
❖ ABSTRACT	2
❖ DECLARATION	3
❖ CERTIFICATE	4-5
❖ ACKNOWLEDGEMENTS	6
CHAPTER 1: INTRODUCTION	8
CHAPTER 2: LITERATURE REVIEW	9
CHAPTER 3: METHODOLOGY	10
3.1 STACK SELECTION	10
3.2 MODULAR DESIGN AND ARCHITECTURE	10
3.3 DEVELOPMENT PROCESS	12
CHAPTER 4: DESIGN AND IMPLEMENTATION	12
4.1 APPLICATION ARCHITECTURE	12
4.2 CORE LOGIC MODULES IMPLEMENTATION	12
4.3 USER INTERFACE (UI) DESIGN	14
4.4 STYLING WITH STYLES.QSS	14
CHAPTER 5: TESTING AND RESULTS	16
5.1 TESTING PROCEDURES	16
5.2 RESULTS ANALYSIS	17
CHAPTER 6: DISCUSSION	18
CHAPTER 7: CONCLUSION	19
REFERENCES	19

CHAPTER 1: INTRODUCTION

In the contemporary landscape of science and engineering, efficient and versatile computational tools are indispensable. While a plethora of calculators exist, most cater exclusively to basic arithmetic or highly specialized functions, often necessitating users to juggle multiple applications for diverse tasks. This fragmentation of tools creates a significant impediment, particularly for students, educators, and professionals working across disciplines like digital electronics, computer science, and fundamental electrical engineering. The repetitive act of switching between a standard calculator for arithmetic, a separate tool for number base conversions, and yet another for logic operations or electrical calculations leads to inefficiencies, increased cognitive load, and potential errors.

This project directly addresses this prevalent limitation by proposing the "Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications." The central motivation is to consolidate these disparate computational functionalities into a single, intuitive, and robust software application.

The primary objectives of this project are:

- To develop a Python-based scientific calculator capable of performing a comprehensive suite of standard arithmetic operations (addition, subtraction, multiplication, division, logarithms, powers, roots).
- To seamlessly integrate number base conversion capabilities, enabling effortless inter-conversion between binary, octal, decimal, and hexadecimal number systems, along with arithmetic operations within specified bases.
- To incorporate essential digital logic operations, including the calculation of 1's and 2's complements for binary numbers, conversion between binary and Gray codes, and the evaluation and truth table generation for various 2-input and 3-input logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR).
- To include dedicated modules for practical electrical engineering calculations, specifically for determining resistance values from standard resistor colour codes (4-band and 5-band configurations) and for applying Ohm's Law to solve for Voltage, Current, Resistance, and Power given any two known parameters.

This project report will detail the systematic approach taken to achieve these objectives, from the selection of programming tools to the modular design, implementation specifics, and

rigorous testing phases. The aim is to demonstrate the feasibility and benefits of an integrated computational environment for a wide range of technical disciplines.

CHAPTER 2: LITERATURE REVIEW

The foundation of this project builds upon established principles in digital systems, number theory, and electrical engineering, alongside advancements in graphical user interface (GUI) development. Traditional scientific calculators, such as those from Texas Instruments or Casio, have long provided advanced arithmetic and trigonometric functions, yet they typically lack integrated features for digital logic or specialized unit conversions relevant to computer science and electronics. Specialized software tools, like logic simulators (e.g., Logisim) or circuit analysis programs (e.g., LTSpice), exist but are often overly complex for simple calculations or lack the general-purpose calculator interface.

The theoretical underpinnings for number base conversions (binary, octal, decimal, hexadecimal) are rooted in positional numeral systems. Concepts such as 1's and 2's complements are fundamental in binary arithmetic for representing signed numbers and simplifying subtraction in digital circuits. Gray codes, known for their single-bit change between successive values, are crucial in error prevention in digital-to-analog converters and other encoders. Ohm's Law ($V=IR$) and the Power Law ($P=VI$) are cornerstones of basic circuit analysis, while resistor colour codes provide a universal method for quickly identifying resistance values in electronic components.

In terms of software development, Python's versatility and extensive libraries, particularly for numerical computation (math module) and GUI development (e.g., PyQt, Tkinter, Kivy), offer a robust platform for creating interactive applications. PyQt6, in particular, provides a mature and feature-rich toolkit for building cross-platform desktop applications with a strong emphasis on widget-based design and a clear Model-View-Controller (MVC) or similar architectural separation potential. This literature review serves as a backdrop, highlighting the existing tools and theoretical knowledge that our integrated calculator aims to leverage and enhance by consolidating functionalities.

CHAPTER 3: METHODOLOGY

The development process for the "Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications" adhered to a structured software engineering methodology, emphasizing modularity, maintainability, and user-centric design.

3.1. Technology Stack Selection

- **Python:** Chosen as the primary programming language due to its readability, extensive standard library, and a vast ecosystem of third-party packages. Its suitability for rapid prototyping and scientific computation made it an ideal choice for the backend logic.
- **PyQt6:** Selected for the graphical user interface (GUI) development. PyQt6 provides a powerful, mature, and cross-platform framework that allows for the creation of rich, interactive desktop applications. Its signal-slot mechanism facilitates clean separation between UI components and underlying logic, contributing to a robust and responsive user experience.
- **QSS (Qt Style Sheets):** Utilized for customizing the visual appearance of the application. QSS, being a CSS-like language, allowed for precise control over widget styling, ensuring a consistent, modern, and aesthetically pleasing interface, as defined in styles.qss.

3.2. Modular Design and Architecture

A core principle guiding this project was the adoption of a highly modular architecture. This involved segmenting the calculator's diverse functionalities into distinct, self-contained Python classes. This approach offers several significant benefits:

- **Separation of Concerns:** Each module is responsible for a specific set of operations (e.g., `BasicCalculatorLogic` for arithmetic, `BaseConverterLogic` for number base conversions), promoting clarity and reducing interdependencies.
- **Maintainability:** Changes or bug fixes within one module are less likely to impact others, simplifying the maintenance process.
- **Scalability:** New functionalities can be added as separate modules with minimal disruption to existing code.
- **Testability:** Individual modules can be unit-tested in isolation, ensuring the correctness of their specific operations.

The `main.py` file orchestrates these modules, with `MainWindow` acting as the central application class, instantiating the various logic classes and managing the PyQt6 UI components. The `QTabWidget` serves as the primary UI container, logically organizing the distinct calculator functionalities into separate tabs.

3.3. Development Process

The development cycle involved:

1. **Requirement Analysis:** Detailed definition of all functionalities, including specific mathematical operations, conversion types, digital logic functions, and electrical calculation requirements.
2. **Design Specification:** High-level design of the overall application architecture, followed by detailed design of each module's methods and data flow. This included planning the UI layout and interaction patterns.
3. **Iterative Implementation:** Each module was implemented incrementally, starting with core functionalities and gradually adding more complex features.
4. **Integrated Testing:** As functionalities were developed, they were subjected to immediate testing to identify and rectify defects early in the development cycle. Both unit tests (for individual logic components) and integration tests (for UI-logic interaction) were performed.
5. **Refinement:** Based on testing results and continuous evaluation, refinements were made to both the logic and the user interface to enhance performance, accuracy, and user experience.

CHAPTER 4: DESIGN AND IMPLEMENTATION

The scientific calculator is implemented as a single PyQt6 application, featuring a multi-tabbed interface for intuitive navigation across its varied functionalities. The design prioritizes user experience through a clean layout, responsive controls, and clear visual feedback, all styled consistently using a custom QSS theme.

4.1. Application Architecture

The overarching architecture is client-side, desktop-based, with the `MainWindow` class in `main.py` serving as the main application entry point. This `MainWindow` instance creates and manages instances of various logic modules and integrates them into a `QTabWidget`. Each tab represents a distinct calculator mode, providing a cohesive yet segmented user experience.

4.2. Core Logic Modules Implementation

The modular design is actualized through dedicated Python classes, each handling specific computational domains:

- ❖ **BasicCalculatorLogic:** This foundational module is responsible for standard arithmetic operations (+, -, *, /), as well as more advanced functions like logarithms (base 10 and natural), power calculations (x^y), and square roots. It also manages a single memory register (M+, M-, MR, MC), allowing users to store and recall values, enhancing usability for multi-step calculations. Robust error handling for critical operations such as division by zero and attempting logarithms of non-positive numbers is built-in, providing informative feedback to the user.
- ❖ **BaseConverterLogic:** This crucial module facilitates inter-conversion between the four primary number bases: Binary (base-2), Octal (base-8), Decimal (base-10), and Hexadecimal (base-16). It incorporates internal methods to convert any given base input to its decimal equivalent first, and then from decimal to the target base, ensuring accuracy and consistency across all conversions. Beyond simple conversions, this module extends its utility by allowing arithmetic operations (addition, subtraction, multiplication, division) to be performed on numbers within a specified input base, displaying the result in the same base. This direct base arithmetic is a significant feature, useful in digital system design and debugging.
- ❖ **UnitConverterLogic:** This general-purpose conversion module broadens the calculator's utility beyond pure numbers. It supports a diverse range of unit conversions across several categories:

- **Length:** Kilometres, meters, centimetres, millimetres, miles, yards, feet, inches.
 - **Mass:** Kilograms, grams, milligrams, pounds, ounces.
 - **Volume:** Liters, millilitres, cubic meters, cubic centimetres, gallons, quarts, pints.
 - **Data Storage:** Bits, bytes, kilobytes, megabytes, gigabytes, terabytes.
 - **Temperature:** Celsius, Fahrenheit, Kelvin. The implementation handles the appropriate conversion factors and formulas for each pair of units within a category, providing quick and accurate results.
- ❖ **DigitalLogicLogic:** Catering specifically to digital electronics and computer science, this module offers essential operations:
- **Complements:** Calculates 1's complement and 2's complement for given binary strings. These operations are fundamental for signed number representation and binary subtraction.
 - **Gray Code Conversion:** Performs conversions between standard binary code and Gray code, useful in situations where single-bit changes are critical to prevent errors.
 - **Logic Gate Evaluation:** Implements the core Boolean logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) for both 2-input and 3-input scenarios, evaluating outputs based on binary inputs.
 - **Truth Table Generation:** A highly valuable feature that automatically generates and displays the truth table for any selected 2-input or 3-input logic gate, providing a quick reference and educational tool. Inputs are typically handled via QCheckBox widgets in the UI to represent binary 0s and 1s.
- ❖ **ResistorCalculatorLogic:** This module is a practical tool for electronics enthusiasts and professionals. It calculates the nominal resistance value and tolerance of resistors based on their colour bands. It supports both common 4-band and 5-band configurations. The implementation includes a ResistorVisualizer custom QWidget, which dynamically draws a visual representation of the resistor, displaying the selected colour bands, providing an intuitive way to verify inputs and understand the output.
- ❖ **OhmsLawCalculatorLogic:** A dedicated module for electrical circuit analysis based on Ohm's Law ($V=IR$) and the Power Law ($P=VI$). Given any two known parameters (Voltage, Current, Resistance, or Power), it accurately calculates the remaining two unknown values. The logic incorporates robust parsing of input strings to handle common engineering prefixes (e.g., 'k' for kilo, 'm' for milli, 'u' for micro), ensuring

real-world applicability and user convenience. Results are displayed with appropriate units and prefixes.

4.3. User Interface (UI) Design

The user interface, developed with PyQt6, is designed for clarity, ease of use, and visual appeal. The MainWindow serves as the container for a QTabWidget, presenting distinct functionalities: "Basic/Scientific," "Base Converter," "Unit Converter," "Digital Logic," "Resistor Calculator," and "Ohm's Law."

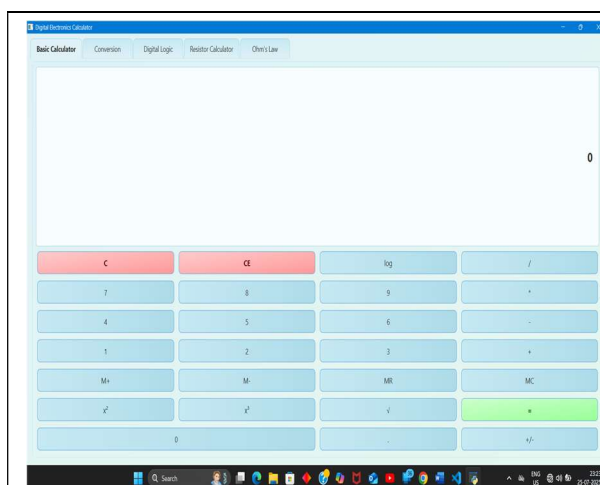
- ❖ **Input and Display:** QLineEdit widgets are extensively used for both input and displaying results across all tabs. These are styled to be clean and prominent, guiding the user's attention.
- ❖ **Buttons:** QPushButton widgets are used for numerical input, operators, and function calls. They are laid out in logical grids using QGridLayout to mimic traditional calculator interfaces, especially in the "Basic/Scientific" tab.
- ❖ **Selection Controls:** QComboBox widgets are employed for selecting bases in the "Base Converter" and units in the "Unit Converter," providing dropdown menus for easy selection. QRadioButton and QCheckBox are used in the "Digital Logic" tab for input selection and gate configuration.
- ❖ **Feedback Labels:** QLabel widgets are strategically placed to display calculation results, error messages, and provide instructional text, ensuring users receive immediate feedback on their operations.

4.4. Styling with styles.qss

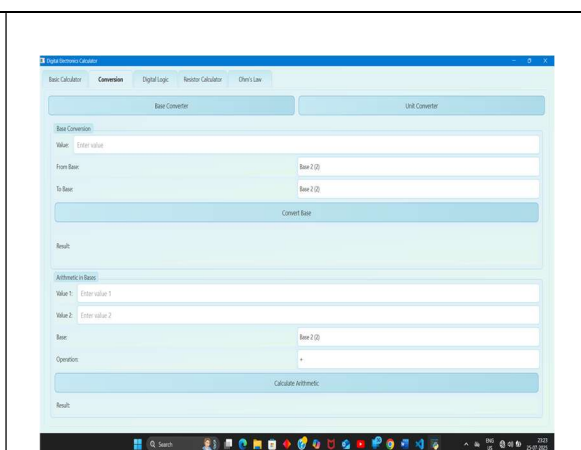
The aesthetic appeal and consistent user experience are significantly enhanced by the application of a custom stylesheet, styles.qss, which defines a "Light Dynamic Gradient Theme."

- **Overall Theme:** The main window (QMainWindow) and general widgets feature a subtle linear background gradient, transitioning from a light blue (#E0F2F7) to a light green (#E6F7F0), creating a fresh and modern feel. The default text colour is a dark grey (#333333) for optimal readability.
- **Tab Styling:** The QTabWidget pane is given a soft blue background and a subtle border. The individual tabs (QTabBar::tab) are styled with rounded top corners and gradient backgrounds that visually change when selected or hovered over, providing clear interactive cues.

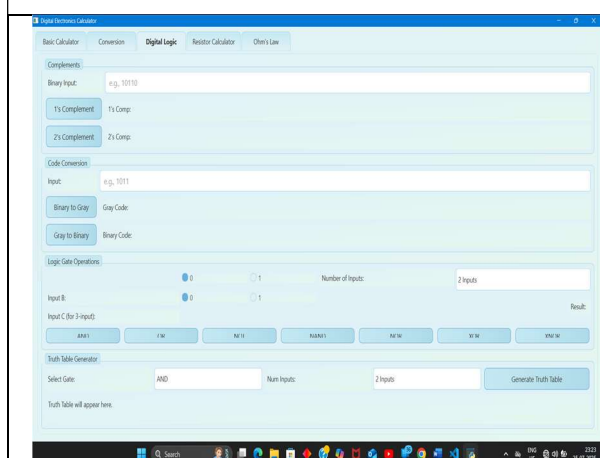
- **Input Fields:** QLineEdit elements have a clean white background with a light blue border, which transitions to a more pronounced accent blue (#6BB0DC) when the field is in focus, guiding user input.
- **Button Aesthetics:** QPushButton elements are designed with linear gradients from soft blues, providing a subtle depth. On hover, these gradients shift to a vibrant accent blue, and on press, they deepen further, offering tactile visual feedback. The text colour on buttons also adjusts to ensure contrast against the changing backgrounds.
- **Radio and Check Boxes:** These controls are styled with clear indicators, using light blue for unchecked states and the accent blue for checked states, ensuring their status is easily discernible.
- **Resistor Visualizer:** The ResistorVisualizer custom widget employs QPainter to dynamically draw the resistor body and bands based on selected colours, leveraging QPen and QBrush for precise rendering.



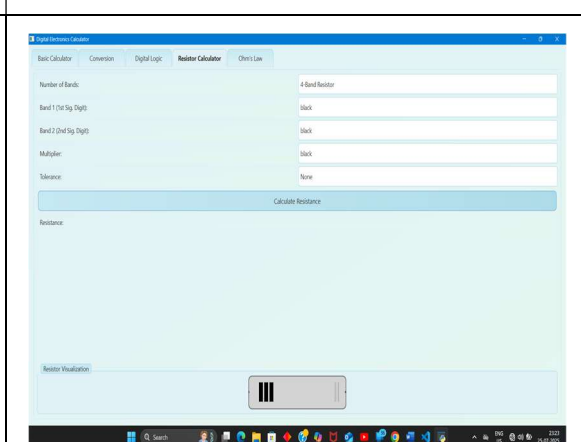
USER INTERFACE 1



USER INTERFACE 2



USER INTERFACE 3



USER INTERFACE 4

Gallary 1: Photos of User Interface of the Calculator

CHAPTER 5: TESTING AND RESULTS

Rigorous testing was an integral part of the development cycle to ensure the calculator's accuracy, reliability, and robustness across all its functionalities. The testing strategy encompassed both unit testing for individual logic modules and integration testing to verify the seamless interaction between the UI and backend components.

5.1. Testing Procedures:

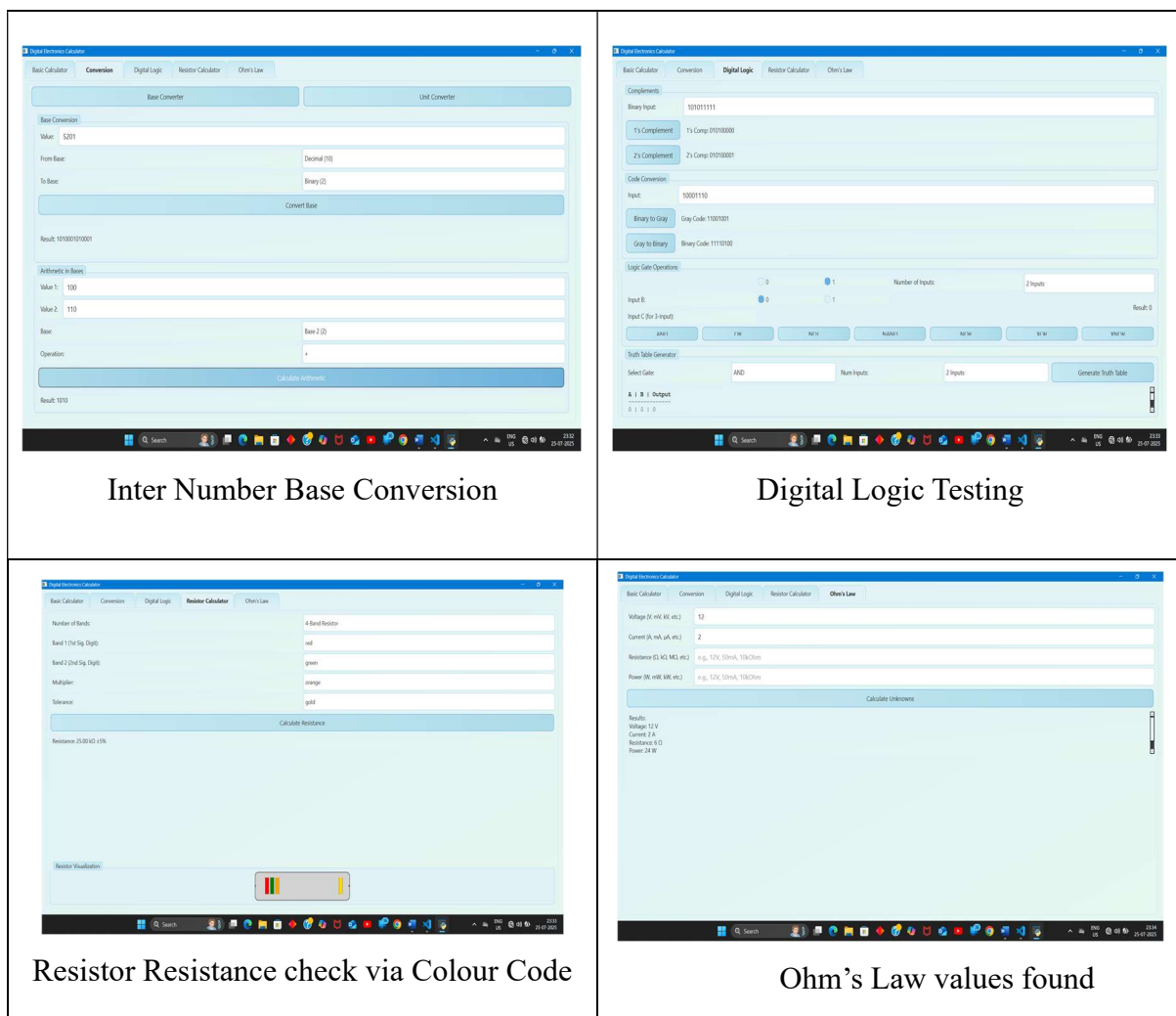
- **Unit Testing:** Each method within BasicCalculatorLogic, BaseConverterLogic, UnitConverterLogic, DigitalLogicLogic, ResistorCalculatorLogic, and OhmsLawCalculatorLogic was tested in isolation. This involved providing a wide range of inputs, including valid, invalid, and edge-case scenarios, and comparing the actual outputs against manually calculated or known correct results.
 - ❖ *Example:* For BaseConverterLogic, testing involved converting large decimal numbers to binary, hexadecimal numbers to octal, and performing arithmetic (e.g., "1011 binary + 10 binary") to ensure correct results in the specified base.
 - ❖ *Example:* For DigitalLogicLogic, truth tables were generated and compared against standard Boolean algebra tables. Complements were checked for various binary string lengths.
 - ❖ *Example:* For OhmsLawCalculatorLogic, inputs like "V=5, R=2" were tested to ensure correct current and power outputs, along with cases involving engineering prefixes (e.g., "V=5V, I=10mA").
- **Integration Testing:** After individual modules were verified, the entire application was tested as a whole. This involved simulating user interactions through the PyQt6 GUI, entering various inputs, and observing the displayed outputs and any error messages. This ensured that UI elements correctly triggered backend computations and that results were displayed accurately.
- **Edge Case and Error Handling Testing:** Specific attention was paid to testing edge cases that could lead to errors or unexpected behaviour:
 - ❖ Division by zero in BasicCalculatorLogic.
 - ❖ Logarithm of non-positive numbers.
 - ❖ Invalid characters or formats for base conversions (e.g., "53G" for binary).
 - ❖ Inputs with too many or too few known values for Ohm's Law calculations.

- ❖ Empty inputs or non-numeric inputs where numbers are expected.

5.2. Results Analysis: The testing process confirmed the successful implementation and reliability of all calculator functionalities.

- **Accuracy:** All arithmetic, conversion, and calculation results matched expected values across diverse test cases, indicating high computational accuracy.
- **Robustness:** The application effectively handles invalid inputs and prevents crashes by displaying clear, user-friendly error messages (e.g., "Division by zero," "Invalid input for base conversion," "Provide exactly two known values" for Ohm's Law).
- **Usability:** The UI demonstrated responsiveness, with buttons and input fields reacting as expected, contributing to an intuitive user experience.

The comprehensive testing regimen ensured that the developed scientific calculator is a dependable and accurate tool for its intended users.



Gallary 2: Photos of Application Testing

CHAPTER 6: DISCUSSION

The "**Design and Implementation of a Scientific Calculator Featuring Number Base Conversions, Complements, Gray Code, and Ohm's Law Applications**" project stands as a successful endeavour in creating a highly versatile and integrated computational tool. The initial problem statement, highlighting the inefficiencies caused by fragmented specialized calculators, has been effectively addressed. By consolidating a broad spectrum of functionalities—from fundamental arithmetic to advanced digital logic operations and practical electrical engineering calculations—into a single application, the project significantly enhances the workflow for its target users.

The architectural decision to employ a modular design, separating UI concerns from computational logic through distinct Python classes, proved invaluable. This approach not only facilitated parallel development and simplified debugging but also established a highly maintainable and scalable codebase. Future expansions or modifications can be implemented with minimal impact on existing functionalities, ensuring the longevity and adaptability of the application. The choice of Python, coupled with the robust PyQt6 framework, allowed for rapid development of a feature-rich application with a sophisticated graphical user interface. The custom styling, applied via `styles.qss`, further elevates the user experience by providing a visually appealing and consistent interface that guides the user intuitively through the various calculator modes.

The integration of specific features like real-time base conversions, comprehensive digital logic analysis including truth tables, and an interactive resistor colour code visualizer, distinguishes this calculator from generic alternatives. The Ohm's Law module, with its intelligent prefix parsing, brings a practical, real-world utility that is often missing in standard calculators. This project demonstrates a strong understanding of both software engineering principles and the underlying mathematical and scientific concepts required for its diverse functionalities.

CHAPTER 7: CONCLUSION

In conclusion, the developed scientific calculator successfully achieves its core objective of providing a unified and comprehensive computational environment. It effectively bridges the gap between conventional arithmetic tools and specialized calculators required for digital electronics, computer science, and basic electrical engineering. The application's ability to seamlessly perform number base conversions, digital logic operations (complements, Gray code, truth tables), and practical electrical calculations (resistor codes, Ohm's Law) within a single, intuitive interface represents a significant advancement. This project not only delivers a highly functional and accurate tool but also showcases the efficacy of modular software design and the power of Python and PyQt6 in developing robust desktop applications. It serves as an efficient resource for students, educators, and professionals, significantly streamlining complex calculations and enriching the understanding of fundamental concepts in digital systems and basic electronics.

References:

- M. M. Mano and M. D. Ciletti. *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*. Pearson, [Latest Edition].
 - T. L. Floyd. *Electronic Devices (Conventional Current Version)*. Pearson, [Latest Edition].
 - M. Summerfield. *Programming Qt with Python: The Definitive Guide to Building GUI Applications on Linux, Mac OS X, and Windows*. Prentice Hall, [Year].
 - Riverbank Computing. *PyQt6 Reference Guide*. [Online]. Available: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>
 - Python Software Foundation. *Python Documentation*. [Online]. Available: <https://docs.python.org/>
 - Real Python. [Online]. Available: <https://realpython.com/>
 - All About Circuits. [Online]. Available: <https://www.allaboutcircuits.com/>
 - GeeksforGeeks. *Digital Electronics*. [Online]. Available: <https://www.geeksforgeeks.org/digital-electronics-logic-design-tutorials/>
-